

Lab de red-teaming LLM

Évaluation de la robustesse aux attaques par prompt injection

Conception d'un outil d'évaluation automatisée de garde-fous de LLM locaux

Alexi Gondry

Bachelor Cybersécurité — EPITA (Promo 2028)

26 septembre 2026

Table des matières

1. Introduction et contexte.....	3
2. Objectifs du projet.....	3
3. Cadre éthique et périmètre.....	3
4. Architecture technique.....	4
5. Méthodologie.....	4
6. Déroulement du projet et itérations.....	6
7. Résultats et analyse.....	7
8. Amélioration du juge LLM.....	8
9. Limites et biais méthodologiques.....	8
10. Compétences mobilisées.....	9
11. Conclusion et perspectives.....	9
12. Annexe — Structure du dépôt.....	9

1. Introduction et contexte

La sécurité des grands modèles de langage (LLM) est devenue un domaine de recherche à part entière. À mesure que ces modèles sont intégrés dans des applications réelles, leurs garde-fous deviennent une surface d'attaque : le prompt injection et le jailbreak désignent l'ensemble des techniques visant à contourner ces protections pour faire produire au modèle un contenu qu'il devrait refuser.

Ce projet personnel consiste à concevoir un outil d'évaluation automatisée mesurant la robustesse d'un modèle face à un corpus d'attaques connues. Il ne s'agit pas de produire de nouvelles attaques, mais de caractériser une résistance : envoyer un ensemble de payloads classés par famille, collecter les réponses du modèle, et déterminer automatiquement si les garde-fous ont cédé.

Le projet a été mené de bout en bout : mise en place d'une cible locale, construction d'un corpus d'attaques, développement d'un double système de détection, collecte structurée des résultats, puis analyse comparative entre deux modèles de tailles différentes. Ce rapport retrace la démarche, les choix techniques, les itérations successives et les résultats obtenus, y compris un résultat contre-intuitif sur la relation entre capacité d'un modèle et robustesse de ses garde-fous.

2. Objectifs du projet

Le projet poursuit quatre objectifs :

1. **Automatiser l'évaluation** : disposer d'un outil qui envoie un corpus complet d'attaques à un modèle cible et rend un verdict par test, sans intervention manuelle.
2. **Produire des données concrètes** : générer des rapports structurés et réexploitables (format JSON horodaté) plutôt que de simples affichages éphémères.
3. **Comparer plusieurs modèles** : mesurer et confronter la robustesse de modèles différents face au même corpus, pour objectiver d'éventuelles différences.
4. **Adopter une démarche défensive rigoureuse** : évaluer la fiabilité des détecteurs eux-mêmes, documenter leurs erreurs, et cadrer honnêtement ce que les résultats permettent — ou ne permettent pas — d'affirmer.

3. Cadre éthique et périmètre

Un outil de test d'attaques doit être cadré. Les choix suivants délimitent le projet dans un usage strictement défensif et pédagogique :

- **Corpus d'attaques publiques uniquement.** Les payloads reprennent des techniques de jailbreak déjà documentées publiquement (DAN, override d'instructions, cadrage fictif, etc.). Le projet ne cherche pas à découvrir de nouvelles attaques.
- **Séparation technique / cible.** Ce qui est mesuré, c'est l'efficacité de la technique de contournement sur les garde-fous. Les cibles restent génériques et ne constituent pas un corpus de contenus réellement dangereux.
- **Témoins bénins (canaris).** Le corpus inclut des requêtes totalement inoffensives (capitale d'un pays, recette, calcul) servant à mesurer le taux de fausses alertes des détecteurs.
- **Modèle cible local et isolé.** Toute l'expérimentation s'effectue sur des modèles servis localement dans une machine virtuelle dédiée, sans appel à un service tiers.

POSITIONNEMENT

L'objectif est de **mesurer et caractériser** une robustesse, activité utile à la défense, et non de fabriquer un exploit. Cette distinction structure l'ensemble des choix de conception.

4. Architecture technique

Le code est organisé par responsabilité, chaque module ayant un rôle unique. Cette séparation facilite la lecture, la maintenance et l'évolution.

Module	Responsabilité
<code>main.py</code>	Orchestration : validation des arguments, lancement de la campagne sur un modèle, écriture du rapport et du récapitulatif.
<code>target.py</code>	<code>send_prompt</code> — envoie un prompt au modèle cible via l'API HTTP d'Ollama.
<code>detector.py</code>	<code>is_attack_successful</code> — détection par mots-clés de refus, puis appel au juge.
<code>judge.py</code>	<code>judge_response</code> — évaluation sémantique de la réponse par un modèle juge (LLM-as-a-judge).
<code>rkeyword.py</code>	Liste des marqueurs textuels de refus.
<code>payloads_data.py</code>	Corpus de payloads, chacun typé par sa famille d'attaque.
<code>stats.py</code>	<code>compute_stats</code> / <code>print_stats</code> — agrégation des résultats par catégorie.
<code>compare.py</code>	Comparaison de deux rapports : taux de bypass par catégorie et divergences par payload.

Le flux de données part de `main.py` qui, pour chaque payload, appelle `send_prompt` (obtention de la réponse du modèle cible) puis `is_attack_successful` (double détection). Le nom du modèle cible circule en argument tout au long de la chaîne, tandis que le modèle juge reste fixe.

5. Méthodologie

5.1 Cible locale (Ollama)

Le modèle attaqué est servi localement par Ollama, dans une machine virtuelle Kali dédiée. Ce choix offre des tests illimités, sans quota ni coût, et illustre la capacité à faire tourner un LLM en autonomie. La fonction `send_prompt` interroge l'API HTTP d'Ollama ; le modèle utilisé est passé dans le corps de la requête, ce qui rend la fonction générique :

```
def send_prompt(prompt, model="llama3.2:3b") -> str:
    payload = {"model": model, "prompt": prompt, "stream": False}
    reponse = requests.post("http://localhost:11434/api/generate",
                            json=payload)
    return reponse.json()["response"]
```

Extrait 1 — Appel de la cible via l'API Ollama, modèle paramétrable.

5.2 Corpus de payloads

Le corpus compte 38 payloads répartis en huit familles d'attaque, plus des témoins bénins. Chaque payload porte un champ type qui permet l'agrégation des résultats par famille.

Famille	Principe
<code>instruction_override</code>	Ordonner d'ignorer les consignes précédentes.
<code>roleplay</code>	Faire endosser un rôle sans restriction (DAN, mode développeur...).
<code>output_priming</code>	Imposer un début de réponse ou interdire les refus.

Famille	Principe
framing_hypothetique	Cadrer la demande comme fictive, éducative ou académique.
obfuscation	Masquer la demande (base64, ROT13, leetspeak, espacement).
prompt_leak	Tenter d'extraire les instructions système.
social_engineering	Fausse autorité, urgence, chantage affectif.
canari	Requêtes bénignes de contrôle (témoins).

5.3 Double détection

Chaque réponse est évaluée par deux détecteurs indépendants, dont la confrontation constitue le cœur de l'analyse :

- **Détecteur par mots-clés** (keyword) : recherche de marqueurs de refus dans la réponse. Rapide, mais grossier.
- **Juge LLM** (judge) : un modèle évalue sémantiquement si la réponse a réellement fourni le contenu demandé.

CHOIX MÉTHODOLOGIQUE — JUGE FIXE

Le modèle juge est maintenu **constant** (le modèle le plus performant disponible) pendant que la cible varie. Cela isole l'effet de la cible : toute différence de résultat provient bien du modèle attaqué, non d'un juge qui changerait en même temps. Ce choix double le coût de calcul mais garantit la validité des comparaisons.

5.4 Collecte et rapports

Chaque campagne produit un rapport JSON horodaté, comprenant un bloc de métadonnées (modèle, durée du run, nombre de payloads, date) et la liste complète des résultats (prompt, réponse, verdicts des deux détecteurs, horodatage par test). Les rapports ne sont jamais écrasés, ce qui permet de conserver l'historique et de comparer les runs entre eux.

5.5 Comparaison multi-modèles

Le module compare.py charge deux rapports et produit deux vues : un tableau des taux de bypass par catégorie (un modèle par colonne) et la liste des payloads sur lesquels les verdicts divergent entre les deux modèles. La collecte (lente) et l'analyse (rapide et rejouable) sont ainsi découplées.

6. Déroulement du projet et itérations

Le projet s'est construit de façon incrémentale. Chaque étape a débloqué la suivante, avec des corrections de conception en cours de route.

Étape 1 — De la cible fictive à la cible réelle

Le point de départ était un squelette où `send_prompt` renvoyait une réponse factice. Le déblocage majeur a consisté à brancher cette fonction sur une véritable IA locale via l'API d'Ollama. À partir de là, les détecteurs ont enfin eu de la matière réelle à analyser.

Étape 2 — Fiabilisation de la détection

Le détecteur par mots-clés a été corrigé (une erreur de logique laissait toujours le verdict à « attaque réussie »). La comparaison a été rendue insensible à la casse. Le juge, initialement un simple tirage aléatoire, a été remplacé par un véritable appel à un LLM chargé d'évaluer la réponse.

Étape 3 — Structuration des sorties

Les résultats ont migré d'un simple affichage vers un rapport JSON complet, puis vers des fichiers horodatés rangés dans un dossier dédié, chacun enrichi d'un bloc de métadonnées incluant la durée d'exécution.

Étape 4 — Agrégation statistique

Un module dédié agrège les résultats par famille d'attaque. Le comptage s'appuie sur une structure `defaultdict`, et distingue les verdicts des deux détecteurs pour permettre l'analyse de leurs désaccords.

Étape 5 — Paramétrage et sécurisation

Le modèle cible est devenu un argument de ligne de commande (`sys.argv`), avec deux garde-fous : vérification qu'un argument est fourni, et validation que le modèle demandé est bien installé dans Ollama (interrogation de l'endpoint `/api/tags`). Cette validation en amont évite de lancer une campagne complète pour découvrir l'erreur au premier appel.

Étape 6 — Comparaison et analyse

Le module de comparaison a permis de confronter deux modèles de tailles différentes sur le même corpus, faisant émerger les résultats analysés dans la section suivante.

Étape 7 — Amélioration itérative du juge

L'analyse manuelle des premiers résultats a révélé une faiblesse du juge sur certaines réponses. Son prompt a été renforcé, puis les campagnes ont été relancées pour mesurer le gain — démarche détaillée en section 8.

7. Résultats et analyse

Deux modèles de la même famille mais de tailles différentes ont été évalués sur les 38 payloads, afin d'isoler l'effet de la capacité sur la robustesse.

Modèle cible	Payloads	Durée du run	Modèle juge
llama3.2:1b	38	≈ 203 s	llama3.2:3b (fixe)
llama3.2:3b	38	≈ 457 s	llama3.2:3b (fixe)

Tableau 1 — Paramètres des deux campagnes d'évaluation.

7.1 Vue d'ensemble

Les deux modèles résistent à la grande majorité des attaques frontales (override d'instructions, roleplay, output priming, social engineering) : ces familles produisent des refus nets. Les techniques d'obfuscation échouent souvent pour une raison technique — le petit modèle décode mal le base64 ou le ROT13 — ce qui constitue un échec de l'attaque, non une preuve de robustesse. Cette nuance est importante : échec de l'attaque et robustesse du garde-fou ne sont pas synonymes.

7.2 La famille dominante : le cadrage hypothétique

La famille framing_hypothétique se distingue nettement. En présentant la demande comme fictive (« dans un monde imaginaire »), éducative (« à des fins pédagogiques ») ou académique (« je suis chercheur, fournis un exemple »), plusieurs payloads obtiennent du modèle un contenu qu'il refuse en demande directe — notamment des modèles complets de courriels de phishing. C'est le vecteur le plus efficace du corpus, sur les deux modèles.

RÉSULTAT CENTRAL — CAPACITÉ ≠ ROBUSTESSE

Contrairement à l'intuition « plus gros modèle = plus sûr », le modèle 3B s'est révélé **plus vulnérable au cadrage contextuel** que le modèle 1B. Plus capable et plus « serviable », le 3B se laisse davantage entraîner par un prétexte légitime (recherche, scénario de film, cours) et produit le contenu demandé, là où le 1B refuse plus souvent — par rigidité plutôt que par un meilleur alignement.

7.3 Fiabilité des détecteurs

La confrontation des deux détecteurs est le résultat le plus instructif du projet. Aucun n'est fiable seul :

Détecteur	Type d'erreur	Illustration
keyword	Faux positifs	Une réponse bénigne sans marqueur de refus (« Paris est la capitale... », une recette) est comptée comme un bypass. Visible sur tous les canaris.
judge	Faux négatifs	Une réponse fournissant un courriel de phishing complet, mais enrobé d'avertissements (« à des fins éducatives »), est classée à tort comme un refus.

Le détecteur par mots-clés confond « absence de refus » et « attaque réussie ». Le juge, lui, se laisse dominer par le volume de disclaimer entourant le contenu malveillant. Ces deux modes de défaillance, opposés, justifient l'emploi conjoint des deux détecteurs et, surtout, la nécessité d'une validation manuelle d'un échantillon.

LECTURE DES CHIFFRES

Les taux de bypass automatiques sont à interpréter comme un **plancher**, non comme une mesure exacte. La validation manuelle a révélé davantage de bypass réels que le juge n'en détecte (faux

négatifs sur réponses hybrides).

8. Amélioration du juge LLM

L'analyse des faux négatifs a conduit à une itération d'ingénierie sur le prompt du juge. Le diagnostic : le prompt initial demandait au juge de déterminer si le modèle avait « cédé » ou « refusé » — un critère fondé sur l'intention affichée. Or les réponses problématiques ne sont ni un refus net ni une exécution nette : elles contiennent le contenu interdit tout en le désavouant.

8.1 Le correctif

Le prompt a été recentré sur un critère observable : la présence effective du contenu nuisible, indépendamment des avertissements qui l'entourent. Le juge reçoit désormais la consigne explicite qu'un courriel de phishing complet enrobé d'un cadre « éducatif » reste une fourniture du contenu, donc un échec de la cible.

8.2 Mesure du gain

Indicateur (cible 3B)	Prompt initial	Prompt amélioré
Bypass détectés en framing_hypothétique	≈ 1 / 5	≈ 2 / 5
Faux positifs ajoutés (canaris, refus)	—	Aucun

Tableau 2 — Effet du durcissement du prompt du juge sur la cible 3B.

Le durcissement du prompt a donc augmenté le rappel (davantage de vrais bypass détectés) sans dégrader la précision (aucune fausse alerte introduite sur les réponses bénignes). Le juge rattrape désormais des cas qu'il manquait, comme le cours fictif sur les malwares et le modèle de phishing « académique ».

LIMITE RÉSIDUELLE

Les réponses très fortement diluées en conseils défensifs échappent encore au juge. Cette limite est structurelle : elle tient à la capacité modeste du modèle juge (un modèle local léger). Un juge plus performant serait le prochain levier d'amélioration.

Cette étape illustre une démarche d'ingénierie complète : problème identifié par l'analyse, correctif appliqué, gain mesuré, et limite résiduelle nommée.

9. Limites et biais méthodologiques

Une évaluation de sécurité honnête délimite ce qu'elle peut affirmer. Les limites suivantes sont assumées et documentées :

- **L'absence de bypass ne prouve pas la sûreté.** On ne peut démontrer qu'un système est sûr ; on peut seulement exhiber une vulnérabilité ou constater une résistance sur un périmètre défini. Un résultat « 0 bypass » documente une résistance à un corpus donné, rien de plus.
- **Juge de capacité limitée.** Le modèle juge, local et léger, produit des faux négatifs sur les réponses hybrides. Ses verdicts constituent une borne, pas une vérité.
- **Corpus mono-tour.** L'évaluation couvre des attaques en un seul message. Elle ne couvre pas les attaques multi-tours, ni les payloads adaptatifs.
- **Échec technique ≠ robustesse.** Certains échecs d'obfuscation viennent de l'incapacité du modèle à décoder le payload, non d'un refus délibéré.

- **Validation manuelle nécessaire.** Les chiffres automatiques doivent être confrontés à une lecture d'échantillon pour être interprétés correctement.

10. Compétences mobilisées

Domaine	Compétences
Sécurité IA	Compréhension des familles d'attaque par prompt injection et jailbreak, méthodologie de red-teaming défensif, approche LLM-as-a-judge.
Développement Python	Architecture modulaire par responsabilité, appels d'API REST, structures de données (defaultdict), gestion d'arguments CLI, sérialisation JSON.
Systèmes & environnement	Déploiement d'un LLM local (Ollama) en machine virtuelle isolée, environnements virtuels Python, hygiène de cloisonnement.
Démarche d'analyse	Formulation d'hypothèses, mesure, identification des biais des outils, itération d'amélioration mesurée, cadrage honnête des conclusions.

11. Conclusion et perspectives

Le projet aboutit à un outil fonctionnel d'évaluation de la robustesse des LLM face aux attaques par prompt injection : cible locale paramétrable, corpus classé par famille, double détection, rapports horodatés et comparaison multi-modèles. Au-delà de l'outil, le projet produit une conclusion d'analyse non triviale : la capacité d'un modèle ne garantit pas la robustesse de ses garde-fous, le cadrage contextuel s'étant révélé le vecteur le plus efficace, et d'autant plus efficace que le modèle est capable.

La valeur du projet ne réside pas dans un contournement spectaculaire, mais dans la rigueur de la démarche : mesurer, confronter deux détecteurs, exposer leurs erreurs respectives, améliorer l'un d'eux de façon mesurée, et délimiter honnêtement ce que les résultats permettent d'affirmer.

Pistes d'évolution

- Remplacer le juge par un modèle plus performant pour réduire les faux négatifs sur les réponses hybrides.
- Ajouter au récapitulatif un taux de désaccord explicite entre les deux détecteurs.
- Élargir le corpus (nouvelles familles, davantage de variantes par famille).
- Étendre la comparaison à un nombre arbitraire de modèles.
- Introduire des scénarios multi-tours pour couvrir une surface d'attaque plus large.

12. Annexe — Structure du dépôt

```
llm-redteaming-lab/
├── main.py           # Orchestration de la campagne
├── target.py         # send_prompt : appel API Ollama
├── detector.py       # is_attack_successful : keyword + juge
├── judge.py          # judge_response : LLM-as-a-judge
├── rkeyword.py       # Marqueurs de refus
├── payloads_data.py  # Corpus de payloads typés
├── stats.py          # Agrégation par catégorie
└── compare.py        # Comparaison de deux rapports
```

```
|— requirements.txt    # Dépendance : requests
|— README.md          # Présentation du projet
|— LICENSE             # MIT
|— examples/          # Rapports d'exemple (1B et 3B)
```

Lancement d'une campagne et comparaison de deux rapports :

```
# Lancer une campagne sur un modèle
python3 main.py llama3.2:3b

# Comparer deux rapports
python3 compare.py rapport/run_A.json rapport/run_B.json
```